



TECNOLÓGICO
NACIONAL DE MÉXICO



TEC DE
JUÁREZ
Forjando el futuro,
transformando vidas.

TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE CIUDAD JUÁREZ

DEPARTAMENTO DE SISTEMAS COMPUTACIONALES

NCIE

Digitalización de Procesos Internos del Nodo: Manual Técnico v2.0

FECHA DE ENTREGA: 11 DE SEPTIEMBRE DE 2026

Gestor:

Edgar David Palacios Flores

Alumnos:

Oscar Chavez Muñoz (v1.0)

Joshua Velasco (v2.0)

Control de Versiones

Versión	Fecha	Autor	Descripción de Cambios
1.0	10 de Marzo, 2026	Oscar Chavez Muñoz	Emisión inicial del manual de arquitectura y especificaciones del sistema NCIE (Apache, MySQL, PHP 8.1 en servidor Ubuntu).
2.0	11 de Septiembre, 2026	Joshua Velasco	Actualización a la arquitectura v2.0: despliegue en contenedores Docker (NGINX 1.27, PHP-FPM 8.2, PostgreSQL 16), nuevo procedimiento de despliegue, validación, respaldos y recuperación; modelo de datos verificado tabla por tabla; roles y permisos; rediseño de portada y pantallas de acceso; interruptor de verificación de correo y correo en modo log; datos de demostración; estado del código y pendientes conocidos.

Documento Confidencial | Propiedad de NCIE | Versión 2.0

Índice

1. Resumen Ejecutivo	5
2. Arquitectura del Sistema	5
2.1. Diagrama de Arquitectura	5
2.2. Componentes Arquitectónicos	6
2.3. Ecosistema Frontend	7
2.4. Configuración de Entorno	8
3. Ingeniería y Modelo de Datos	9
3.1. Módulo de Gestión de Identidad (IAM)	9
3.2. Módulo de Personas (perfiles)	10
3.3. Módulo Operativo – Nodo Académico	10
3.4. Módulo de Proyectos (Innovación / Emprendimiento)	11
3.5. Módulo de Comunicación (Avisos y Notificaciones)	11
3.6. Diccionario de Tablas	11
4. Directrices Estándar de Mantenimiento	12
5. Procedimiento de Despliegue (v2.0, Docker)	13
5.1. Paso 1: Preparación del equipo anfitrión	13
5.2. Paso 2: Obtención del proyecto	13
5.3. Paso 3: Configuración del entorno (.env)	13
5.4. Paso 4: Construcción y arranque de los contenedores	14
5.5. Paso 5: Dependencias PHP y clave de la aplicación	14
5.6. Paso 6: Base de datos	14
5.7. Paso 7: Recursos estáticos y permisos de escritura	14
5.8. Paso 8: Optimización para producción	15
5.9. Paso 9: Exposición pública y HTTPS	15
5.10. Paso 10: Integración con servicios externos (correo)	15
5.11. Despliegue alternativo sin contenedores (resumen)	15

6. Validación del Despliegue	16
6.1. Estado de los contenedores	16
6.2. Accesibilidad local	16
6.3. Acceso externo	16
6.4. Verificación de logs	16
6.5. Validación funcional	16
7. Mantenimiento Operativo	16
8. Problemas Comunes y Soluciones	17
9. Plan de Rollback y Recuperación ante Desastres	18
9.1. Fallos durante el Despliegue Inicial (rollback por instalación)	18
9.2. Protocolos de Actuación por Incidente (casos críticos)	18
10. Política de Respaldos Cíclicos (Backups Automáticos)	19
11. Buenas Prácticas de Producción	19
12. Estado del Código y Pendientes Conocidos	20
13. Anexo A: Cuentas de Demostración	21
14. Anexo B: Variables de Entorno	21
15. Anexo C: Inventario de Rutas	22
16. Anexo D: Referencia Rápida de Comandos	23

1. Resumen Ejecutivo

El presente Manual Técnico consolida la documentación de arquitectura, componentes, modelo de datos, despliegue, operación y recuperación de la plataforma NCIE (Nodo de Creatividad, Innovación y Emprendimiento) del Instituto Tecnológico de Ciudad Juárez. Está dirigido a mantenedores de software, personal de infraestructura y líderes técnicos responsables de instalar, operar, respaldar y evolucionar el sistema.

Esta **versión 2.0** sustituye a la 1.0 (marzo de 2026), que describía un despliegue directo sobre un servidor Ubuntu con Apache, PHP 8.1 y MySQL. La versión 1.0 anticipaba una segunda fase con PostgreSQL y un servidor web estandarizado; esa fase es la que aquí se documenta. Los cambios principales respecto a la versión anterior son:

- **Despliegue en contenedores Docker** (`docker-compose.yml`): NGINX 1.27 como servidor web, PHP-FPM 8.2 con Laravel 10.48 y **PostgreSQL 16** como motor de base de datos, con volumen persistente, comprobación de salud y política de reinicio automático.
- **Nuevo procedimiento de despliegue, validación, mantenimiento, respaldo y recuperación** basado en Docker Compose, y un procedimiento alternativo resumido para servidores sin contenedores.
- **Modelo de datos verificado tabla por tabla** contra las migraciones y la base de datos real (25 tablas, 20 migraciones, 14 modelos), con el catálogo de roles y permisos (5 roles, 105 permisos).
- **Rediseño de la portada pública y de las pantallas de acceso** con un sistema visual propio (sin Bootstrap en la portada), áreas y horarios de atención cargados desde la base de datos, y re-tematización del panel administrativo (AdminLTE 3).
- **Configuración operativa**: correo saliente en modo `log` para el entorno local, interruptor de verificación de correo (NCIE_EMAIL_VERIFICATION), datos de demostración (DemoSeeder) y cuentas de prueba por rol.
- **Correcciones** respecto a la auditoría de agosto de 2026 (ruta de notificaciones leídas, importación inexistente, horarios en la portada) y un **registro de pendientes conocidos** con su ubicación en el código.

Todos los datos técnicos de este manual (versiones, puertos, conteos de tablas, rutas y permisos, comportamiento de las funciones) fueron verificados el 11 de septiembre de 2026 sobre el árbol de trabajo `~/Nodo/public_html` y el stack Docker en ejecución (Docker Engine 29.6.2 en macOS 26.6). La Bitácora de Pruebas v2.0 documenta la re-ejecución de las pruebas funcionales sobre esta misma versión.

2. Arquitectura del Sistema

El sistema NCIE está construido sobre el framework **Laravel 10** (versión instalada 10.48.29) bajo el patrón MVC y se ejecuta, a partir de la versión 2.0, como un conjunto de tres contenedores orquestados con Docker Compose: servidor web, aplicación PHP y base de datos. El código fuente vive en el equipo anfitrión y se monta dentro de los contenedores, de modo que un cambio en el código es visible de inmediato sin reconstruir imágenes.

2.1. Diagrama de Arquitectura

Flujo de operación: Usuario → Navegador → NGINX (`localhost:8080`) → PHP-FPM / Laravel (`app:9000`) → PostgreSQL (`db:5432`). NGINX entrega directamente los archivos estáticos de `public/` y reenvía cualquier otra petición al front controller `public/index.php` mediante FastCGI. La base de datos persiste en el volumen Docker `pgdata`.

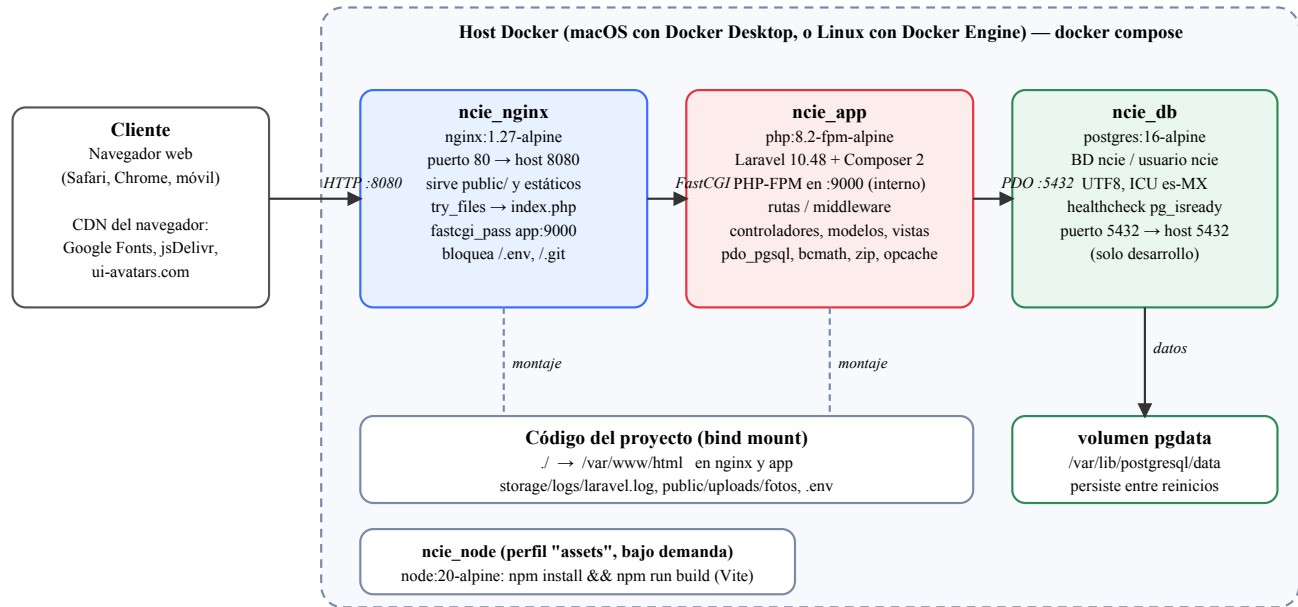


Figura 1: Diagrama de arquitectura v2.0 del Sistema NCIE (contenedores, puertos y montajes).

2.2. Componentes Arquitectónicos

Capa de infraestructura (contenedores). Definida en `docker-compose.yml`, `docker/php/Dockerfile` y `docker/nginx/default.conf`. Los tres servicios permanentes llevan `restart: unless-stopped`, por lo que vuelven a levantarse solos cuando arranca Docker; el servicio `app` espera a que la base de datos reporte estado `healthy` antes de iniciar.

Servicio / contenedor	Imagen (versión verificada)	Puertos	Función y archivos relacionados
nginx / ncie_nginx	nginx:1.27-alpine (1.27.5)	80 → host 8080	Servidor web. Raíz <code>/var/www/html/public</code> ; <code>try_files</code> hacia <code>index.php</code> ; <code>client_max_body_size 20M</code> ; bloquea archivos ocultos (<code>/.env</code> , <code>/.git</code>) y oculta la cabecera <code>X-Powered-By</code> . Configuración: <code>docker/nginx/default.conf</code> .
app / ncie_app	php:8.2-fpm-alpine (PHP 8.2.33) construida desde <code>docker/php/Dockerfile</code> ; Composer 2.10	9000 (solo red interna)	PHP-FPM con la aplicación Laravel. Extensiones añadidas: <code>pdo_pgsql</code> , <code>bcmath</code> , <code>zip</code> , <code>opcache</code> (el resto viene en la imagen oficial). Aquí se ejecutan <code>composer</code> y <code>php artisan</code> .
db / ncie_db	postgres:16-alpine (16.15)	5432 → host 5432 (uso de desarrollo)	Base de datos <code>ncie</code> , usuario <code>ncie</code> . Inicializada con <code>--encoding=UTF8 --locale-provider=icu --icu-locale=es-MX</code> para que el orden alfabético respete acentos y ñ. Healthcheck <code>pg_isready</code> cada 5 s. Datos en el volumen <code>pgdata</code> .
node / ncie_node	node:20-alpine (perfil <code>assets</code>)	—	Se ejecuta bajo demanda (<code>docker compose --profile assets run --rm node</code>) para compilar los assets de Vite (<code>npm install && npm run build</code>). No forma parte del stack en ejecución.

Consumo observado en reposo (11-sep-2026): `ncie_app` 59 MiB, `ncie_db` 21 MiB y `ncie_nginx` 8 MiB de memoria; las tres imágenes ocupan 1.26 GB en disco y el volumen de datos de demostración 47 MB.

Capa de aplicación (Laravel). Se conserva la estructura MVC descrita en la versión 1.0:

- **Enrutamiento y middleware:** `routes/web.php` (115 definiciones con verbo HTTP) más las rutas de autenticación que registra `Auth::routes(['verify' => true])` de `laravel/ui`; en total 128 rutas web. Las rutas del panel llevan la cadena de middleware `auth, verified` y `can:<nombre-de-ruta>`; 101 rutas están protegidas por un permiso de `spatie/laravel-permission` cuyo nombre coincide con el nombre de la ruta.
- **Controladores:** 26 archivos en `app/Http/Controllers/` (20 de dominio y 6 de autenticación en `Auth/`). Contienen la validación y la lógica de negocio; no existen `Form Requests` ni políticas (`app/Http/Requests` y `app/Policies` no existen).
- **Modelos:** 14 modelos Eloquent en `app/Models/` (ver sección 3). Solo `User` define *casts* (`email_verified_at` como fecha y `password` como *hashed*).
- **Vistas:** 99 plantillas Blade en `resources/views/` (82 propias y 17 de correo en `vendor/`); 65 vistas del panel extienden `layouts/admin.blade.php`, la portada es `index.blade.php` y las pantallas de acceso extienden `layouts/auth.blade.php`.
- **Eventos y notificaciones:** `PostEvent` → `PostListener` → `PostNotification` (canal *database*) para los avisos internos; `SugerenciaMail` para el buzón de sugerencias de la portada.
- **Base de datos y semillas:** 20 migraciones en `database/migrations/` y tres seeders (`RoleSeeder`, `DatabaseSeeder`, `DemoSeeder`).

Los módulos funcionales, su prefijo de ruta y los roles que los usan se resumen en la siguiente tabla (el Anexo C detalla el inventario de rutas).

Módulo	Prefijo de rutas	Rutas	Roles con acceso
Portada pública y horarios por área (AJAX)	<code>/, /areas/{id}</code>	2	Público
Autenticación, registro, recuperación y verificación de correo	<code>/login, /register, /password/*, /email/*</code>	17	Público / autenticado
Buzón de sugerencias (correo al nodo)	<code>POST /sugerencias/enviar</code>	1	Usuario registrado y verificado
Panel: tablero con totales y calendario de cursos	<code>/admin, /admin/cursos/calendar-events</code>	2	Todos los autenticados
Usuarios y Administrativos	<code>/admin/usuarios, /admin/administracion</code>	16	admin
Alumnos, Áreas, Gestores, Horarios	<code>/admin/alumnos, /admin/areas, /admin/gestores, /admin/horarios</code>	33	admin, administrativo
Cursos, Asignaciones gestor–curso	<code>/admin/cursos, /admin/asignaciones</code>	15	admin, administrativo
Proyectos, Proyecto–gestores, Alumno–proyecto, Presupuestos	<code>/admin/proyectos, /admin/proyecto_gestores, /admin/alumno_proyecto, /admin/presupuestos</code>	29	admin, administrativo
Inscripciones y "Mis cursos"	<code>/inscripciones/*, /mis-cursos</code>	4	alumno, usuario
"Mis proyectos" (alumno)	<code>/mis-proyectos</code>	1	alumno
Reportes del gestor (cursos, inscritos, proyectos)	<code>/reportes/*</code>	3	gestor
Avisos y notificaciones	<code>/post/*, /markAsRead, /mark-as-read</code>	5	Crear: admin, administrativo, gestor; leer: todos

2.3. Ecosistema Frontend

La versión 2.0 abandona la plantilla "Medilab" y Bootstrap 5 en la parte pública. La capa de presentación se divide en tres conjuntos de recursos, todos servidos por NGINX desde `public/` sin proceso de compilación obligatorio:

- **Portada pública y pantallas de acceso** (`resources/views/index.blade.php`, `layouts/auth.blade.php` y las siete vistas de `auth/`): hoja de estilos propia `public/assets/css/ncie.css` (837 líneas) con un sistema de tokens en `:root` (fondo `#070A14/#0B1230`, rojo `#E5323F`, azul `#4F7DFF`, superficies de vidrio, radios de `28/22/16` px) y la familia tipográfica **Archivo** (Google Fonts). Solo se conservan como dependencias `bootstrap-icons` y `glightbox` (en `public/assets/vendor/`). Las áreas del nodo se leen de la tabla `areas` y el horario de atención de cada área se carga por AJAX desde `GET /areas/{id}` (vista `cargar_datos_areas.blade.php`). El registro incluye un diálogo nativo `<dialog>` con los términos y condiciones.
- **Panel administrativo** (`layouts/admin.blade.php` y vistas de `admin/`, `inscripciones/`, `reportes/`, `post/`): AdminLTE 3 sobre Bootstrap 4 (`public/dist`, `public/plugins`), Font Awesome, DataTables con los complementos Responsive y Buttons (exportación a Excel, CSV, PDF, copia e impresión mediante JSZip y pdfmake), SweetAlert2 (CDN), FullCalendar 6.1.18 (CDN, con el idioma español local en `public/fullcalendar/es.global.js`) y avatares generados por `ui-avatars.com`. El tema visual del panel se aplica al final con `public/dist/css/ncie-admin.css` (tokens: primario `#3B6CFF`, azul marino `#0B1230`, rojo `#E5323F`), que oscurece la barra lateral y la barra superior y redondea los controles; el área de contenido de los CRUD permanece clara.
- **Vite** (`vite.config.js`, `resources/sass/app.scss`, `resources/js/app.js` → `public/build/`): solo lo usan los layouts `app` y `app2` (vista `home`), que no forman parte de los flujos principales. El build actual está incluido en `public/build/`; puede regenerarse con el servicio `node` del Compose. No debe existir el archivo `public/hot` (marcador del servidor de desarrollo de Vite), pues haría que esas vistas intentaran cargar recursos desde `http://[::1]:5173`.

El navegador del usuario necesita salida a internet para las fuentes y bibliotecas servidas por CDN (`fonts.googleapis.com`, `cdn.jsdelivr.net`, `ui-avatars.com`); el servidor no la necesita en operación normal.

2.4. Configuración de Entorno

La configuración se lee de `.env` (nunca versionado). El archivo `.env.docker` es la plantilla para el stack de contenedores y `.env.example` la plantilla genérica de Laravel. Las variables relevantes y sus valores verificados en el entorno local se listan en el Anexo B; las más importantes son:

APP_ENV / APP_DEBUG	<code>local / true</code> en el entorno de desarrollo. En producción deben ser <code>production / false</code> .
APP_URL	<code>http://localhost:8080</code> . Cambiar al dominio o IP pública cuando se exponga el sistema.
DB_CONNECTION, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD	<code>pgsql, db</code> (nombre del servicio en Compose), <code>5432, ncie, ncie</code> y la contraseña definida en <code>POSTGRES_PASSWORD</code> de <code>docker-compose.yml</code> (<code>ncie_password</code> por defecto; cambiar en producción).
MAIL_MAILER	<code>log</code> : los correos (verificación, restablecimiento de contraseña, sugerencias) se escriben en <code>storage/logs/laravel.log</code> . Para envío real usar <code>smtp</code> con credenciales válidas (<code>MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_ENCRYPTION, MAIL_FROM_ADDRESS</code>).
NCIE_EMAIL_VERIFICATION	Interruptor propio definido en <code>config/ncie.php</code> (<code>ncie.email_verification</code>). Con <code>false</code> (valor actual) las cuentas creadas por autoregistro quedan verificadas al instante y entran directo al panel; con <code>true</code> se envía el correo de verificación y se exige confirmarlo. Tras cambiarlo ejecutar <code>php artisan config:clear</code> .
SESSION_DRIVER, CACHE_DRIVER, QUEUE_CONNECTION	<code>file, file, sync</code> : no se requieren Redis ni workers de cola.

LOG_CHANNEL / LOG_LEVEL

stack / debug en local; en producción se recomienda daily y error.

Zona horaria de la aplicación: UTC (config/app.php); idioma: es (traducciones en lang/es/). Las marcas de tiempo de laravel.log están en UTC.

3. Ingeniería y Modelo de Datos

La base de datos se rige por integridad referencial con borrado en cascada (18 claves foráneas). Las 20 migraciones crean 25 tablas: 19 con nombre literal (5 base de Laravel/Sanctum y 14 del dominio, fechadas entre mayo y agosto de 2025) y 5 del paquete spatie/laravel-permission. Todas las migraciones se ejecutan sin cambios sobre PostgreSQL 16; el único ajuste de comportamiento es que el enum de alumno_proyecto.tipo se crea como varchar con restricción CHECK. El modelo se organiza en cinco bloques lógicos (Figura 2).

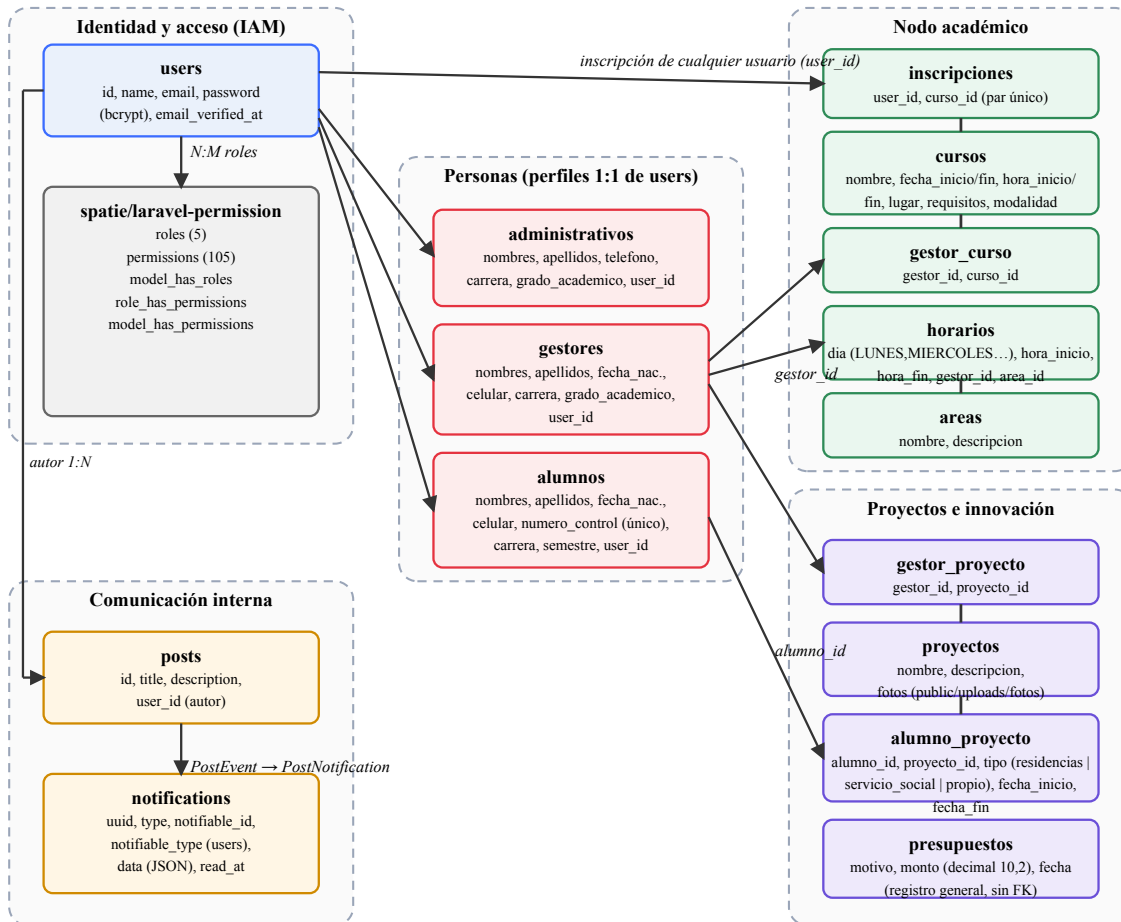


Figura 2: Modelo de datos del Sistema NCIE v2.0 (tablas, columnas principales y relaciones).

3.1. Módulo de Gestión de Identidad (IAM)

users (modelo User, implementa MustVerifyEmail): repositorio de autenticación con name, email (único), password (Bcrypt mediante el cast hashed), email_verified_at y remember_token. Un usuario se especializa opcionalmente en un perfil 1:1 de administrativos, gestores o alumnos; los usuarios que se registran desde la portada no tienen perfil y reciben el rol usuario.

Roles y permisos (spatie/laravel-permission 6.20): tablas `roles`, `permissions`, `model_has_roles`, `model_has_permissions` y `role_has_permissions`. `RoleSeeder` es la fuente de verdad: crea 5 roles y 105 permisos con el mismo nombre que la ruta que protegen (por ejemplo `admin.cursos.index`). El usuario administrador inicial lo crea `DatabaseSeeder`.

Rol	Permisos	Alcance funcional
admin	96	Acceso total. Único rol que gestiona Usuarios (<code>/admin/usuarios</code>) y Administrativos (<code>/admin/administracion</code>).
administrativo	80	Operación del nodo: alumnos, áreas, gestores, horarios, cursos, asignaciones, proyectos, vínculos de proyectos y presupuestos; publica avisos.
gestor	5	Reportes de sus cursos e inscritos (<code>/reportes</code> , <code>/reportes/cursos-asignados</code>), sus proyectos (<code>/reportes/mis_proyectos</code>) y publicación de avisos.
alumno	5	Inscribirse a cursos, ver y abandonar "Mis cursos", ver "Mis proyectos".
usuario	4	Rol por defecto del autoregistro: inscribirse a cursos y ver "Mis cursos".

El permiso `admin.index` existe pero no está asignado a ningún rol (el tablero `/admin` solo exige `auth` y `verified`); y cuatro permisos sembrados no tienen ruta que los use (`admin.asignaciones.show`, `admin.proyecto_gestores.edit`, `admin.proyecto_gestores.update`, `admin.index`). Al añadir una ruta protegida con `can`: debe añadirse el permiso correspondiente en `RoleSeeder` y ejecutar `php artisan permission:cache-reset`.

3.2. Módulo de Personas (perfiles)

- administrativos** (modelo `Administrativo`): nombres, apellidos, telefono, carrera, grado_academico, `user_id` (FK a `users`, cascada). Se crea junto con su usuario desde `/admin/administracion` y recibe el rol `administrativo`.
- gestores** (modelo `Gestor`, tabla `gestores`): nombres, apellidos, fecha_nacimiento (texto), celular (10), carrera, grado_academico, `user_id`. Relaciones: `horarios` (1:N), `cursos` (N:M vía `gestor_curso`), `proyectos` (N:M vía `gestor_proyecto`). La relación `area()` declarada en el modelo no tiene columna en la tabla (la vinculación gestor-área se materializa a través de `horarios`).
- alumnos** (modelo `Alumno`): nombres, apellidos, fecha_nacimiento (texto), celular, numero_control (único, hasta 8 caracteres), carrera, semestre, `user_id`. Relación `proyectos` (N:M vía `alumno_proyecto` con tipo, `fecha_inicio`, `fecha_fin`).

Al eliminar un perfil desde el panel, el controlador elimina también el `User` asociado; no existe baja lógica (*soft delete*).

3.3. Módulo Operativo – Nodo Académico

- areas** (modelo `Area`): nombre (100), descripcion (500). Se muestran en la portada y agrupan los horarios de atención.
- horarios** (modelo `Horario`): horario de atención de un gestor en un área (no el horario de los cursos, como decía la v1.0): día guarda los días seleccionados separados por coma (LUNES,MIERCOLES), `hora_inicio`, `hora_fin`, `gestor_id`, `area_id` (ambas con cascada). El controlador impide que un gestor tenga más de un horario.
- cursos** (modelo `Curso`): nombre, `fecha_inicio`, `fecha_fin`, `hora_inicio`, `hora_fin`, lugar, requisitos, modalidad (`presencial` | `en_linea`), descripcion. Se validan fechas futuras y coherentes. No existe campo de cupo.

- **gestor_curso** (modelo `GestorCurso`): asignación gestor ↔ curso (`gestor_id`, `curso_id`); al crearla se verifica que el gestor no tenga otro curso con fechas y horas solapadas.
- **inscripciones** (modelo `Inscripcion`): `user_id`, `curso_id`, par único. Cualquier usuario autenticado con permiso (`alumno` o `usuario`) puede inscribirse a cursos que tengan gestor asignado, no hayan terminado y no hayan empezado hace más de dos días; se rechazan duplicados y choques de horario con otros cursos ya inscritos. El usuario puede abandonar el curso desde "Mis cursos".

3.4. Módulo de Proyectos (Innovación / Emprendimiento)

- **proyectos** (modelo `Proyecto`): `nombre` (300), `descripcion` (400), `fotos` (ruta relativa `uploads/fotos/<timestamp>_<archivo>`; el archivo se guarda con `move()` en `public/uploads/fotos`, no en `storage/`). Al eliminar el proyecto se borra la imagen.
- **gestor_proyecto** (modelo `GestorProyecto`): responsables o supervisores del proyecto (`gestor_id`, `proyecto_id`; el controlador evita duplicados).
- **alumno_proyecto** (modelo `AlumnoProyecto`): participantes (`alumno_id`, `proyecto_id`, `tipo` ∈ {`residencias`, `servicio_social`, `propio`}, `fecha_inicio`, `fecha_fin`). Permite N alumnos por proyecto con modalidades distintas.
- **presupuestos** (modelo `Presupuesto`): `motivo` (texto), `monto` (decimal 10,2), `fecha`. Es un registro contable general del nodo: **no** está ligado a proyectos (no tiene clave foránea).

3.5. Módulo de Comunicación (Avisos y Notificaciones)

- **posts** (modelo `Post`): avisos internos con `title`, `description` y `user_id` (autor). Solo existen rutas de creación y consulta; no hay edición ni borrado.
- **notifications**: tabla estándar de Laravel (`uuid`, `type`, `notifiable_type/notifiable_id` hacia `users`, `data` JSON con el aviso, `read_at`). Al publicar un aviso, `PostListener` envía una `PostNotification` a cada usuario de los roles destino (`admin`, `alumno`, `gestor`, `administrativo`; o solo `usuario` si se marca "Solo usuarios externos"), excluyendo al autor. La campana del panel muestra las no leídas; `POST /mark-as-read` las marca como leídas y devuelve JSON.

3.6. Diccionario de Tablas

Tabla	Columnas (además de id y timestamps)	Relaciones y notas
users	name, email (único), email_verified_at, password, remember_token	hasOne administrativos/gestores/alumnos; hasMany inscripciones, posts; morphMany notifications
roles / permissions / model_has_roles / role_has_permissions / model_has_permissions	name, guard_name; claves compuestas en las pivote	spatie/laravel-permission; 5 roles, 105 permisos
administrativos	nombres, apellidos, telefono, carrera, grado_academico, user_id	FK user_id → users (cascade)
gestores	nombres, apellidos, fecha_nacimiento (string), celular(10), carrera, grado_academico, user_id	FK user_id → users (cascade)
alumnos	nombres, apellidos, fecha_nacimiento (string), celular, numero_control (único), carrera, semestre, user_id	FK user_id → users (cascade)
areas	nombre(100), descripcion(500)	hasMany horarios
horarios	día (CSV de días), hora_inicio, hora_fin, gestor_id, area_id	FK gestor_id → gestores, area_id → areas (cascade)
cursos	nombre, fecha_inicio, fecha_fin, hora_inicio, hora_fin, lugar, requisitos?, modalidad, descripcion?	belongsToMany gestores (gestor_curso), users (inscripciones)

Tabla	Columnas (además de id y timestamps)	Relaciones y notas
gestor_curso	gestor_id, curso_id	FK con cascade; sin restricción de unicidad en BD (la valida el controlador)
inscripciones	user_id, curso_id	FK con cascade; único (user_id, curso_id)
proyectos	nombre(300), descripcion(400), fotos?	belongsToMany gestores (gestor_proyecto), alumnos (alumno_proyecto)
gestor_proyecto	gestor_id, proyecto_id	FK con cascade
alumno_proyecto	alumno_id, proyecto_id, tipo (residencias servicio_social propio), fecha_inicio?, fecha_fin?	FK con cascade; enum → varchar + CHECK en PostgreSQL
presupuestos	motivo (text), monto decimal(10,2), fecha	Sin claves foráneas
posts	title, description (text), user_id	FK user_id → users (sin cascade)
notifications	id uuid, type, notifiable_type, notifiable_id, data (text/JSON), read_at	Notificaciones de base de datos de Laravel
password_reset_tokens	email (PK), token, created_at	Usada por la recuperación de contraseña
password_resets	email, token, created_at	Tabla legada creada por una migración duplicada; no se usa (candidata a eliminarse)
failed_jobs, personal_access_tokens, migrations	estándar de Laravel / Sanctum	Sanctum instalado pero sin uso funcional (solo GET /api/user)

4. Directrices Estándar de Mantenimiento

Todo miembro del equipo de ingeniería debe adherirse a los siguientes estándares para el mantenimiento correctivo y evolutivo:

- Cambios en la base de datos:** jamás modificar la estructura mediante SQL directo. Todo cambio se traza con una migración (`php artisan make:migration ...`) ejecutada dentro del contenedor (`docker compose exec app php artisan migrate`). Las migraciones deben funcionar en PostgreSQL (evitar `enum`, funciones o tipos exclusivos de MySQL).
- Validación de peticiones:** mover la validación voluminosa de los controladores a Form Requests (`app/Http/Requests`, que actualmente no existe) conforme se toquen los módulos.
- Consultas N+1:** usar *eager loading* (`with()`) en los controladores para las vistas que recorren relaciones; `preventLazyLoading` no está activado.
- Registro y monitoreo:** los fallos se registran en `storage/logs/laravel.log` (hora UTC). Es el primer archivo que debe consultarse al depurar, junto con `docker compose logs app nginx db`.
- Compatibilidad con PostgreSQL:** las comparaciones de texto distinguen mayúsculas (normalizar los correos con `strtolower` al guardar y buscar) y el orden de `Model::all()` es indefinido (añadir `orderBy()` explícito en los listados). Las fechas deben almacenarse como `date`, no como texto.
- Roles y permisos:** `RoleSeeder` es la fuente de verdad. Toda ruta nueva del panel se protege con `can:<nombre.de.ruta>`, se siembra el permiso con el mismo nombre y se asigna a los roles correspondientes; después `php artisan permission:cache-reset`.
- Interruptores de configuración:** las banderas propias del sistema viven en `config/ncie.php` y se leen con `config('ncie.*')`; nunca consultar `env()` fuera de los archivos de configuración.

8. **Frontend:** la portada y las pantallas de acceso se ajustan modificando los tokens de `public/assets/css/ncie.css`; no reintroducir Bootstrap ni la plantilla Medilab en la portada. En el panel, `ncie-admin.css` debe cargarse siempre al final para sobrescribir a AdminLTE.
9. **Pruebas:** ejecutar `docker compose exec app php artisan test` antes de entregar cambios. La suite actual solo contiene las pruebas de ejemplo; cada corrección debería añadir una prueba de característica (Feature test) del flujo afectado.
10. **Control de versiones:** el repositorio `Joshua-Velasco/NCIEPortal` (GitHub) contiene la versión de abril de 2026 y no incluye los cambios de la v2.0 (Docker, DemoSeeder, rediseño). Debe subirse el árbol actual sin versionar `.env`, `vendor/`, `node_modules/`, `storage/logs/`, `public/uploads/fotos/` ni respaldos de base de datos.

5. Procedimiento de Despliegue (v2.0, Docker)

El procedimiento siguiente instala el sistema desde cero en un equipo con Docker. Es el mismo tanto para el entorno de desarrollo/demostración (macOS con Docker Desktop) como para un servidor Linux; las diferencias de producción se señalan en cada paso. Todos los comandos se ejecutan desde la raíz del proyecto.

5.1. Paso 1: Preparación del equipo anfitrión

1. **Docker:** en macOS o Windows instalar Docker Desktop 4.x (verificado 4.85 / Engine 29.6); en Ubuntu Server 22.04+ instalar Docker Engine 24+ con el complemento Compose v2 (`sudo apt install docker-ce docker-ce-cli containerd.io docker-compose-plugin`) y añadir el usuario operador al grupo `docker`.
2. **Recursos:** 2 vCPU y 4 GB de RAM mínimos; 8 GB recomendados. En Macs de 8 GB, Docker Desktop reserva por defecto 4 GB para su máquina virtual; conviene bajar ese valor a 3 GB en *Settings* → *Resources* y cerrar aplicaciones pesadas antes de arrancarlo (ver sección 8).
3. **Git** para clonar el repositorio (o copiar el árbol del proyecto por otro medio). No es necesario instalar PHP, Composer, Node ni PostgreSQL en el anfitrión.
4. Verificar: `docker --version`, `docker compose version` y `docker info` (debe responder sin error; Docker Desktop tarda entre 30 y 60 s en estar listo tras abrirse).

5.2. Paso 2: Obtención del proyecto

```
git clone git@github.com:Joshua-Velasco/NCIEPortal.git ncie # o copiar el árbol ~/Nodo/public_html
cd ncie
ls docker-compose.yml docker/php/Dockerfile docker/nginx/default.conf .env.docker
```

Estructura de los archivos de contenedores:

```
docker-compose.yml      # servicios app, nginx, db (+ node bajo perfil "assets"), volumen pgdata
docker/php/Dockerfile    # php:8.2-fpm-alpine + pdo_pgsql, bcmath, zip, opcache + Composer 2
docker/nginx/default.conf # servidor NGINX apuntando a public/ y reenviando .php a app:9000
.env.docker              # plantilla de variables de entorno para Docker (pgsql, host "db")
```

5.3. Paso 3: Configuración del entorno (.env)

```
cp .env.docker .env
# Editar .env: al menos APP_URL, DB_PASSWORD, MAIL_* y NCIE_EMAIL_VERIFICATION
```

Variable	Desarrollo / demostración	Producción
APP_ENV / APP_DEBUG	local / true	production / false (obligatorio)

Variable	Desarrollo / demostración	Producción
APP_URL	http://localhost:8080	URL pública con https:// (dominio o IP)
DB_CONNECTION, DB_HOST, DB_PORT	pgsql, db, 5432	Igual
DB_DATABASE, DB_USERNAME, DB_PASSWORD	ncie, ncie, ncie_password	Contraseña fuerte, coherente con POSTGRES_PASSWORD en docker-compose.yml
MAIL_MAILER	log	smtp con credenciales institucionales (MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_ENCRYPTION, MAIL_FROM_ADDRESS, MAIL_FROM_NAME)
NCIE_EMAIL_VERIFICATION	false	true cuando el correo saliente funcione
LOG_CHANNEL / LOG_LEVEL	stack / debug	daily / error

Las contraseñas de docker-compose.yml (POSTGRES_PASSWORD) y del .env deben coincidir. En producción, además, no publique el puerto 5432 del servicio db (elimine la línea "5432:5432" o cámbiela por "127.0.0.1:5432:5432").

5.4. Paso 4: Construcción y arranque de los contenedores

```
docker compose up -d --build      # construye la imagen PHP la primera vez y levanta db, app y nginx
docker compose ps                 # ncie_db debe mostrar (healthy); ncie_app y ncie_nginx "Up"
```

La construcción descarga las imágenes base (php:8.2-fpm-alpine, nginx:1.27-alpine, postgres:16-alpine, aproximadamente 1.3 GB en total con la imagen construida) y requiere salida a internet solo en este paso y en la instalación de dependencias.

5.5. Paso 5: Dependencias PHP y clave de la aplicación

```
docker compose exec app composer install --optimize-autoloader      # añadir --no-dev en producción
docker compose exec app php artisan key:generate                    # genera APP_KEY en .env
```

5.6. Paso 6: Base de datos

```
docker compose exec app php artisan migrate --seed                  # 20 migraciones + RoleSeeder + usuario admin
docker compose exec app php artisan db:seed --class=DemoSeeder      # OPCIONAL: datos de demostración (ver Anexo)
```

migrate --seed crea el esquema, los 5 roles, los 105 permisos y el usuario administrador sistemancie@gmail.com con contraseña 12345678 (**cámbiela de inmediato en producción** desde /admin/usuarios). DemoSeeder es re-ejecutable: elimina todos los datos del dominio y todos los usuarios que no sean administradores antes de sembrar 7 áreas, 7 gestores con horario, 3 administrativos, 12 alumnos, 4 usuarios externos, 10 cursos, inscripciones, 8 proyectos con foto, 10 presupuestos, 4 avisos y sus notificaciones; las fechas de los cursos son relativas al día de ejecución.

Migración desde la base MySQL de la versión 1.0 (opcional): crear primero el esquema con migrate --seed; copiar solo los datos con pgloader (WITH data only, reset sequences) o con volcados transformados; si la herramienta no reinicia las secuencias, ejecutar SELECT setval(pg_get_serial_sequence('<tabla>', 'id'), COALESCE(MAX(id), 1)) FROM <tabla>; por cada tabla con id autoincremental; finalmente php artisan permission:cache-reset.

5.7. Paso 7: Recursos estáticos y permisos de escritura

```
docker compose --profile assets run --rm node                      # OPCIONAL: recompila Vite (layouts app/app2)
docker compose exec app chmod -R ug+rw storage bootstrap/cache public/uploads/fotos
```

```
docker compose exec app php artisan view:clear
```

PHP-FPM debe poder escribir en `storage/`, `bootstrap/cache/` y `public/uploads/fotos/` (fotos de proyectos). No se usa `php artisan storage:link` porque el sistema no guarda archivos en `storage/app/public`.

5.8. Paso 8: Optimización para producción

```
docker compose exec app php artisan config:cache
docker compose exec app php artisan route:cache
docker compose exec app php artisan view:cache
# tras cualquier cambio de .env o de rutas: php artisan optimize:clear y repetir
```

5.9. Paso 9: Exposición pública y HTTPS

El stack publica HTTP en el puerto 8080 del anfitrión. Para un servidor institucional se recomienda un proxy inverso en el anfitrión que termine TLS y reenvíe al 8080 (por ejemplo NGINX con Certbot/Let's Encrypt, o Caddy, que obtiene el certificado automáticamente):

```
# /etc/nginx/sites-available/ncie.conf (anfitrión)
server {
    listen 80;
    server_name ncie.example.mx;           # dominio o IP pública
    location / {
        proxy_pass      http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
# sudo certbot --nginx -d ncie.example.mx (añade el bloque 443/TLS)
# sudo ufw allow OpenSSH && sudo ufw allow 'Nginx Full' && sudo ufw enable
```

Al operar detrás de un proxy, fijar `APP_URL` con `https://` y confiar en el proxy en `app/Http/Middleware/TrustProxies.php` (`protected $proxies = '*';` o la IP del proxy) para que Laravel genere enlaces `https`.

5.10. Paso 10: Integración con servicios externos (correo)

El sistema envía correo para la verificación de cuenta, el restablecimiento de contraseña y el buzón de sugerencias de la portada (destinatario `sistemancie@gmail.com`, fijado en `SugerenciaController`). Con `MAIL_MAILER=log` ningún correo sale del servidor. Para activar el envío: configurar `MAIL_MAILER=smtp` y las credenciales SMTP proporcionadas por el área de TI, poner `NCIE_EMAIL_VERIFICATION=true` si se desea exigir verificación y ejecutar `php artisan config:clear`. Las credenciales SMTP de Gmail usadas en 2025 ya no autentican y no deben reutilizarse.

5.11. Despliegue alternativo sin contenedores (resumen)

Si el servidor institucional no permite Docker, el sistema puede desplegarse de forma tradicional con los mismos componentes: **PHP 8.2** (el `composer.lock` fija paquetes que exigen 8.2, aunque `composer.json` declare `^8.1`) con las extensiones `pdo_pgsql`, `bcmath`, `zip`, `mbstring`, `xml`, `curl`, `tokenizer`, `ctype`, `fileinfo`, `openssl`; **Composer 2**; **PostgreSQL 16** (crear la base `ncie` con `ENCODING 'UTF8' LOCALE_PROVIDER icu ICU_LOCALE 'es-MX' TEMPLATE template0`); **NGINX + PHP-FPM** usando `docker/nginx/default.conf` como referencia (o Apache con el `.htaccess` estándar de Laravel y `DocumentRoot` en `public/`); y **Node.js 20** solo si se recompila Vite. Los pasos de aplicación son los mismos: `composer install`, `.env`, `key:generate`, `migrate --seed`, permisos de `storage/`, `bootstrap/cache/` y `public/uploads/fotos/`, y cachés de optimización. MySQL 8 sigue siendo

técnicamente compatible (DB_CONNECTION=mysql), pero la versión 2.0 se validó únicamente sobre PostgreSQL.

6. Validación del Despliegue

6.1. Estado de los contenedores

```
docker compose ps
# NAME          STATUS          PORTS
# ncie_app       Up ...          9000/tcp
# ncie_db        Up ... (healthy) 0.0.0.0:5432->5432/tcp
# ncie_nginx     Up ...          0.0.0.0:8080->80/tcp
```

6.2. Accesibilidad local

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/ # 200 (portada)
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/login # 200
curl -s -o /dev/null -w "%{http_code} %{redirect_url}\n" http://localhost:8080/admin # 302 → /login
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/.env # 403 (bloqueado por NGINX)
```

6.3. Acceso externo

Abrir desde otro equipo `http://<dominio-o-IP>/` (a través del proxy del Paso 9) y comprobar que la portada muestra las áreas del nodo y que el enlace "Ingresar" lleva a la pantalla de acceso. Si se configuró HTTPS, verificar que el certificado es válido y que los recursos se cargan sin advertencias de contenido mixto.

6.4. Verificación de logs

```
docker compose logs -f nginx app db # salida de los tres servicios
tail -f storage/logs/laravel.log # errores de la aplicación (hora UTC)
docker compose exec db psql -U ncie -d ncie -c "select count(*) from users;"
```

6.5. Validación funcional

Confirmar como mínimo los siguientes puntos (la Bitácora de Pruebas v2.0 contiene la lista completa de 59 casos):

1. Inicio de sesión con el administrador y carga del tablero `/admin` con los totales y el calendario de cursos.
2. La portada lista las áreas registradas y, al elegir una, muestra el horario de atención de sus gestores.
3. Alta, edición y baja de un registro en Áreas y en Cursos; exportación a Excel/PDF desde el botón "Reportes" de un listado.
4. Registro público de una cuenta y entrada al panel (con `NCIE_EMAIL_VERIFICATION=false`) o recepción del correo de verificación (con `true`).
5. `docker compose exec app php artisan test` termina con todas las pruebas en verde.

7. Mantenimiento Operativo

Tareas cotidianas y periódicas para mantener el sistema estable:

Tarea	Comando / procedimiento	Frecuencia
Arranque tras reiniciar el equipo	Abrir Docker Desktop (<code>open -a Docker</code> en macOS) y esperar 30–60 s; los tres contenedores se levantan solos por la política <code>unless-stopped</code> . Si no: <code>docker compose up -d</code> .	Cada arranque
Apagado	<code>docker compose down</code> (conserva la base de datos en el volumen). Nunca <code>docker compose down -v</code> salvo que se quiera destruir la base de datos.	Según necesidad

Tarea	Comando / procedimiento	Frecuencia
Ejecutar Artisan / Composer	<code>docker compose exec app php artisan <comando>; docker compose exec app composer <comando></code>	Según necesidad
Limpieza de cachés	<code>docker compose exec app php artisan optimize:clear</code> (config, rutas, vistas, caché de aplicación). Tras editar vistas: <code>view:clear</code> (las vistas están en caché).	Tras cambios
Revisión de logs	<code>storage/logs/laravel.log</code> y <code>docker compose logs --since 24h app nginx db</code> . Activar <code>LOG_CHANNEL=daily</code> para rotación automática.	Diaria
Consola de base de datos	<code>docker compose exec db psql -U ncie -d ncie</code>	Según necesidad
Reiniciar la demostración	<code>docker compose exec app php artisan db:seed --class=DemoSeeder</code> (borra y vuelve a sembrar; conserva al administrador).	Antes de una demo
Actualizar imágenes base	<code>docker compose pull && docker compose build --pull app && docker compose up -d</code>	Mensual
Actualizar dependencias PHP	<code>docker compose exec app composer update</code> en un entorno de prueba, <code>composer audit</code> , ejecutar la suite y desplegar.	Trimestral
Espacio en disco de Docker	<code>docker system df</code> ; <code>docker builder prune</code> (en el equipo de desarrollo se observaron 4.2 GB de caché de construcción reclamables); <code>docker image prune</code> .	Mensual
Validación de respaldos	Restaurar el último <code>pg_dump</code> en una base temporal y contar registros (sección 10).	Mensual

8. Problemas Comunes y Soluciones

Síntoma	Causa probable	Solución
Docker no arranca o tarda mucho; <code>docker info</code> falla	Docker Desktop no está abierto (no tiene inicio automático) o el equipo está bajo presión de memoria (en un Mac de 8 GB la VM reserva 4 GB).	<code>open -a Docker</code> y esperar 30–60 s; cerrar aplicaciones pesadas; reducir la memoria de la VM a 3 GB en <i>Settings</i> → <i>Resources</i> . Comprobar <code>sysctl vm.swapusage</code> .
Contenedores en estado <i>Exited</i> tras reiniciar	Compose sin política de reinicio (corregido en v2.0 con <code>restart: unless-stopped</code>).	<code>docker compose up -d</code> . Verificar con <code>docker inspect ncie_app --format '{{.HostConfig.RestartPolicy.Name}}'</code> .
<code>port is already allocated</code> al levantar	Los puertos 8080 o 5432 están ocupados por otro servicio.	Cambiar el mapeo en <code>docker-compose.yml</code> (" <code>8081:80</code> ") y <code>APP_URL</code> ; o detener el otro servicio.
Error 500 al registrarse o al pedir restablecer contraseña	Envío SMTP con credenciales inválidas (Failed to authenticate on SMTP server).	En local usar <code>MAIL_MAILER=log</code> ; en producción credenciales SMTP válidas. Luego <code>php artisan config:clear</code> .
Error 500 "Target class [admin.proyecto_gestores.edit] does not exist"	Bug conocido: las rutas <code>edit/update</code> de <code>proyecto_gestores</code> declaran el permiso sin el prefijo <code>can:</code> (<code>routes/web.php</code>).	Corregir a <code>'can:admin.proyecto_gestores.edit'</code> y <code>'can:admin.proyecto_gestores.update'</code> ; <code>route:clear</code> .
La fecha de inicio de un curso no se actualiza al editar	Error tipográfico <code>feha_inicio</code> en <code>CursoController::update</code> .	Corregir la clave a <code>fecha_inicio</code> en el arreglo de <code>update()</code> .
Error 419 <i>Page Expired</i> al enviar un formulario	Token CSRF o sesión caducados (sesión de 120 min en archivos).	Recargar la página y reenviar; verificar que <code>storage/framework/sessions</code> sea escribible.
Error 500 genérico; <i>Permission denied</i> en logs	PHP-FPM no puede escribir en <code>storage/</code> o <code>bootstrap/cache/</code> .	<code>docker compose exec app chmod -R ug+rw storage bootstrap/cache public/uploads/fotos</code> .
No application encryption key has been specified	Falta <code>APP_KEY</code> en <code>.env</code> .	<code>docker compose exec app php artisan key:generate</code> y <code>config:clear</code> .

Síntoma	Causa probable	Solución
Cambios en vistas Blade no se reflejan	Vistas compiladas en caché.	<code>docker compose exec app php artisan view:clear.</code>
Las vistas <code>home/app</code> intentan cargar <code>http://[::1]:5173</code>	Existe <code>public/hot</code> (marcador del servidor de desarrollo de Vite).	Borrar <code>public/hot</code> ; recompilar con el servicio <code>node</code> si faltan archivos en <code>public/build/</code> .
Una cuenta creada desde el panel no puede entrar: la app redirige a <code>/email/verify</code>	Los CRUD de administrativos, gestores y alumnos no marcan la cuenta como verificada ni envían el correo; el interruptor <code>NCIE_EMAIL_VERIFICATION</code> solo cubre el autoregistro.	Que el usuario pulse "reenviar" en <code>/email/verify</code> (con correo funcionando) o marcar <code>email_verified_at</code> en la base de datos; o corregir los controladores para que verifiquen al crear (pendiente P-08).
El login no encuentra un correo escrito con mayúsculas	PostgreSQL distingue mayúsculas en las comparaciones de texto.	Escribir el correo exactamente como se registró; a mediano plazo normalizar a minúsculas al guardar y buscar (pendiente P-06).
<code>duplicate key value violates unique constraint</code> tras importar datos	Secuencias desincronizadas después de una importación con IDs explícitos.	Ejecutar <code>setval</code> por tabla (sección 5, Paso 6).
Listados en orden distinto entre recargas	Consultas sin <code>ORDER BY</code> (<code>Model::all()</code>); en PostgreSQL el orden es indefinido.	Añadir <code>orderBy('id')</code> o por nombre en los <code>index()</code> (pendiente P-07).

9. Plan de Rollback y Recuperación ante Desastres

9.1. Fallos durante el Despliegue Inicial (rollback por instalación)

- **Detener servicios:** `docker compose down` (mantiene el volumen de datos).
- **Imagen defectuosa:** reconstruir sin caché con `docker compose build --no-cache app && docker compose up -d`.
- **Base de datos corrupta o migraciones a medias:** en un entorno nuevo, `docker compose exec app php artisan migrate:fresh --seed` (destruye y recrea el esquema); en un entorno con datos, restaurar el último respaldo (sección 10) o `php artisan migrate:rollback` del lote fallido.
- **Cachés inconsistentes:** `php artisan optimize:clear`.
- **Dependencias en conflicto:** `docker compose exec app rm -rf vendor && composer install`; para los assets `docker compose --profile assets run --rm node`.
- **Volver al estado limpio absoluto (solo laboratorio):** `docker compose down -v` elimina también la base de datos; después repetir los Pasos 4 a 7.

9.2. Protocolos de Actuación por Incidente (casos críticos)

Caso A: ciberataque o acceso no autorizado.

1. **Aislamiento:** `docker compose stop nginx` (el sistema deja de responder de inmediato) y bloquear el puerto en el firewall del anfitrión; revocar accesos SSH sospechosos.
2. **Revocación de credenciales:** cambiar `POSTGRES_PASSWORD/DB_PASSWORD`, las contraseñas de los usuarios administradores y regenerar `APP_KEY` (`php artisan key:generate`), lo que invalida todas las sesiones y cookies emitidas.
3. **Restauración limpia:** obtener el código desde el repositorio en un directorio nuevo, reconstruir la imagen (`docker compose build --no-cache`) y no reutilizar contenedores ni imágenes anteriores.

4. **Recuperación de datos:** restaurar el último respaldo de la base de datos anterior a la intrusión y la copia de `public/uploads/fotos`.
5. **Análisis:** conservar `storage/logs/laravel.log`, los logs de NGINX (`docker compose logs nginx`) y los del anfitrión para la investigación.

Caso B: fallo físico del servidor (corte eléctrico, disco).

1. Instalar Docker en un equipo de reemplazo (Paso 1) y copiar el árbol del proyecto, el archivo `.env` respaldado, el último volcado de la base de datos y la carpeta `public/uploads/fotos`.
2. Ejecutar los Pasos 4, 5 y 7; en lugar de `migrate --seed`, restaurar el volcado: `docker compose exec -T db pg_restore -U ncie -d ncie --clean --if-exists < ncie_<fecha>.dump` (crear la base vacía antes con `migrate` si el volcado no la incluye).
3. Redirigir el DNS o el proxy al nuevo anfitrión y validar según la sección 6. Con respaldos diarios, el tiempo objetivo de recuperación es inferior a una hora y la pérdida máxima de datos es de un día.

10. Política de Respaldos Cíclicos (Backups Automáticos)

- **Alcance:** (1) base de datos PostgreSQL completa; (2) archivos subidos `public/uploads/fotos/`; (3) el archivo `.env` y `docker-compose.yml` (contienen credenciales; guardarlos cifrados o con permisos restringidos).
- **Ejecución:** tarea `cron` diaria en el anfitrión a las 02:00 (franja de inactividad):

```
#!/usr/bin/env bash # /usr/local/bin/ncie-backup.sh
set -euo pipefail
cd /ruta/al/proyecto
DEST=/var/backups/ncie; mkdir -p "$DEST"; F=$(date +%F)
docker compose exec -T db pg_dump -U ncie -Fc ncie > "$DEST/ncie_${F}.dump"
tar czf "$DEST/uploads_${F}.tgz" public/uploads/fotos
cp .env "$DEST/env_${F}"; chmod 600 "$DEST"/env_*
find "$DEST" -type f -mtime +7 -delete # retención local de 7 días
rsync -a "$DEST"/ usuario@nas.institucional:/respaldos/ncie/ # réplica fuera del servidor
# crontab -e → 0 2 * * * /usr/local/bin/ncie-backup.sh >> /var/log/ncie-backup.log 2>&1
```

- **Retención:** 7 días locales con rotación automática y réplica inmediata en almacenamiento externo (NAS institucional o disco externo); conservar además un respaldo mensual durante un año.
- **Restauración:** `docker compose exec -T db pg_restore -U ncie -d ncie --clean --if-exists < ncie_<fecha>.dump` y `tar xzf uploads_<fecha>.tgz`. Alternativa de volumen completo: `docker run --rm -v public_html_pgdata:/data -v $PWD:/backup alpine tar czf /backup/pgdata_<fecha>.tgz /data` (con la base detenida).
- **Verificación:** una vez al mes restaurar el respaldo en una base temporal (`createdb -U ncie ncie_prueba + pg_restore -d ncie_prueba`) y comparar conteos de `users`, `cursos` y `proyectos` con producción.

11. Buenas Prácticas de Producción

- **Modo debug desactivado:** `APP_DEBUG=false` y `APP_ENV=production`; de lo contrario los errores exponen rutas, variables y consultas.
- **Credenciales por defecto:** cambiar la contraseña del administrador sembrado (12345678) y la de PostgreSQL (`ncie_password`) antes de exponer el sistema; no versionar `.env` y protegerlo con `chmod 600`.
- **Red:** exponer solo 80/443 a través del proxy con HTTPS (Let's Encrypt); no publicar el puerto 5432 fuera del anfitrión; mantener el firewall (`ufw`) con el mínimo de puertos.

- **Separación de entornos:** mantener un `.env` por entorno a partir de `.env.docker`; usar `NCIE_EMAIL_VERIFICATION=true` y correo real en producción.
- **Limitación de tasa:** el login está protegido (5 intentos por minuto) y la API a 60 peticiones por minuto, pero las rutas web públicas no tienen límite; si el sistema se expone a internet conviene limitar tasa en el proxy (`limit_req` de NGINX) o con `throttle` de Laravel.
- **Registros:** `LOG_CHANNEL=daily`, limitar el tamaño de los logs de contenedores (`logging: driver: json-file, options: max-size: 10m, max-file: "5"` en Compose) y revisar los errores semanalmente.
- **Actualizaciones:** aplicar parches del sistema operativo y de Docker, renovar imágenes base y ejecutar `composer audit` periódicamente; probar en un entorno de pruebas antes de producción.
- **Respaldos y restauración probados:** seguir la política de la sección 10 y documentar cada prueba de restauración.
- **Usuario operador sin privilegios de root:** administrar el anfitrión con un usuario estándar con `sudo` y acceso SSH por clave (`PermitRootLogin no`).
- **Monitoreo:** `docker stats`, el healthcheck de la base de datos y una comprobación externa periódica de `GET /` (código 200).

12. Estado del Código y Pendientes Conocidos

Resultado de la auditoría de agosto de 2026 y de la verificación del 11 de septiembre de 2026. Los elementos "Corregido" ya están en la versión 2.0; los "Pendiente" siguen presentes y se recomienda atenderlos en el orden indicado por su impacto.

ID	Descripción	Ubicación	Impacto	Estado
C-01	Ruta POST <code>/mark-as-read</code> definida como cadena <code>'PostController@markNotification'</code> (fallaba con 500) e importación de un <code>NotificationController</code> inexistente.	<code>routes/web.php</code> , <code>PostController::markNotification</code>	Notificaciones no se podían marcar como leídas	Corregido (responde JSON <code>{ok:true}</code>)
C-02	Los horarios de atención de los gestores no aparecían en la portada pública.	<code>index.blade.php</code> , <code>cargar_datos_areas.blade.php</code> , <code>WebController</code>	Observación de la Bitácora v1.0	Corregido (tabla por área vía AJAX)
C-03	Contenedores quedaban detenidos tras reiniciar el equipo.	<code>docker-compose.yml</code>	Disponibilidad	Corregido (<code>restart: unless-stopped</code>)
C-04	Error 500 al registrar cuentas por credenciales SMTP inválidas.	<code>.env</code> (<code>MAIL_MAILER=log</code>), <code>config/ncie.php</code>	Registro bloqueado	Corregido (correo en log; verificación opcional)
P-01	Error tipográfico <code>feha_inicio</code> : la fecha de inicio de un curso no se guarda al editar.	<code>app/Http/Controllers/CursoController.php</code> (método <code>update</code>)	Alto: datos incorrectos	Pendiente
P-02	Rutas <code>admin.proyecto_gestores.edit/update</code> con el permiso sin prefijo <code>can</code> : → HTTP 500 al editar una asignación de gestor a proyecto.	<code>routes/web.php</code>	Alto: función inutilizable	Pendiente
P-03	<code>GET /admin/cursos/calendar-events</code> solo exige <code>auth</code> (sin <code>verified</code> ni permiso).	<code>routes/web.php</code>	Bajo: expone el calendario a cualquier sesión	Pendiente
P-04	Migración duplicada <code>create_password_resets_table</code> (tabla legada sin uso).	<code>database/migrations/2014_10_12_100000_*</code>	Bajo	Pendiente

ID	Descripción	Ubicación	Impacto	Estado
P-05	fecha_nacimiento almacenada como texto en alumnos y gestores.	Migraciones de alumnos y gestores	Medio: comparaciones de fechas fallan en PostgreSQL	Pendiente
P-06	Correos no normalizados a minúsculas (login, buzón y validación unique distinguen mayúsculas en PostgreSQL).	SugerenciaController, controladores de personas, modelo User	Medio	Pendiente
P-07	Listados sin ORDER BY (Model::all()): orden indefinido en PostgreSQL.	Métodos index() de los controladores, WebController, AdminController	Bajo	Pendiente
P-08	Cuentas creadas desde el panel (administrativos, gestores, alumnos, usuarios) no quedan verificadas ni reciben correo de verificación; no pueden usar el panel hasta verificar.	AdministrativoController, GestorController, AlumnoController, UsuarioController	Alto en operación diaria	Pendiente
P-09	Los avisos (posts) no pueden editarse ni eliminarse; los usuarios creados en /admin/usuarios no reciben rol; no hay cupo en cursos.	PostController, UsuarioController, modelo Curso	Funcional (mejoras)	Pendiente
P-10	Ruta POST /email/verification-notification definida dos veces (prevalece la segunda, con throttle:6,1); relaciones muertas Administrativo::curso() y Gestor::area(); Sanctum instalado sin uso.	routes/web.php, modelos	Bajo (limpieza)	Pendiente
P-11	Fotos de proyectos en public/uploads/fotos (fuera de storage/); deben incluirse en los respaldos.	ProyectoController	Operativo	Documentado
P-12	Suite de pruebas mínima (2 pruebas de ejemplo) y repositorio GitHub desactualizado (abril de 2026, CI fallando por PHP 8.1 vs. paquetes 8.2).	tests/, Joshua-Velasco/NCIEPortal	Proceso	Pendiente
P-13	El archivo DOCUMENTACION.md del árbol contiene credenciales en texto plano y describe Hostinger/MySQL como producción.	DOCUMENTACION.md, INFO.md	Seguridad documental	Pendiente (depurar antes de versionar)

13. Anexo A: Cuentas de Demostración

Creadas por DatabaseSeeder (administrador) y DemoSeeder (resto). Contraseña 12345678 en todas; los correos ya están verificados. Solo para entornos de desarrollo y demostración.

Rol	Correo	Perfil
admin	sistemancie@gmail.com	Administrador (sin perfil)
administrativo	admin.ncie1@itcj.edu.mx (y admin.ncie2, admin.ncie3)	Personal administrativo
gestor	gestor1@itcj.edu.mx ... gestor7@itcj.edu.mx	Un gestor por área, cada uno con horario de atención
alumno	121110001@cdjuarez.tecnm.mx ... 121110012@cdjuarez.tecnm.mx	Alumnos con inscripciones y proyectos
usuario	laura.beltran@gmail.com (y otros 3 usuarios externos)	Comunidad registrada desde la portada

14. Anexo B: Variables de Entorno

Variable	Valor en el entorno local (verificado)	Notas
APP_NAME	"Sistema NCIE"	Nombre mostrado en correos y títulos
APP_ENV / APP_DEBUG	local / true	production / false en producción
APP_KEY	(generada con key:generate)	Base64; regenerarla invalida sesiones y datos cifrados
APP_URL	http://localhost:8080	Debe coincidir con el puerto publicado por nginx
LOG_CHANNEL / LOG_LEVEL	stack / debug	daily / error recomendado en producción
DB_CONNECTION / DB_HOST / DB_PORT	pgsql / db / 5432	db es el nombre del servicio en Compose
DB_DATABASE / DB_USERNAME / DB_PASSWORD	ncie / ncie / (ver docker-compose.yml)	Coherentes con POSTGRES_* del servicio db
BROADCAST_DRIVER / CACHE_DRIVER / QUEUE_CONNECTION / SESSION_DRIVER	log / file / sync / file	Sin Redis ni colas; SESSION_LIFETIME 120 min
FILESYSTEM_DISK	local	No se usa el disco public ni storage:link
MAIL_MAILER	log	smtp en producción
MAIL_HOST / MAIL_PORT / MAIL_ENCRYPTION	smtp.gmail.com / 587 / tls	Valores heredados; sustituir por el SMTP institucional
MAIL_FROM_ADDRESS / MAIL_FROM_NAME	sistemancie@gmail.com / "Sistema NCIE"	Remitente de los correos del sistema
NCIE_EMAIL_VERIFICATION	false	Interruptor propio (config/ncie.php); true exige verificación
Variables VITE, PUSHER, AWS, REDIS y MEMCACHED_HOST	valores por defecto de Laravel	Sin uso en el sistema

15. Anexo C: Inventario de Rutas

Resumen de `routes/web.php` (115 definiciones con verbo: 72 GET, 18 POST, 12 PUT, 13 DELETE) más las rutas de `laravel/ui`. Cada módulo CRUD sigue el patrón `index, create, store, show, edit, update, confirmDelete` y `destroy`.

Grupo	Rutas (nombre base)	Middleware	Observaciones
Público	<code>index (/)</code> , <code>cargar_datos_areas (/areas/{id})</code> , <code>home</code>	ninguno / auth	<code>/home</code> redirige tras login de <code>laravel/ui</code> ; el sistema usa <code>/admin</code>
Autenticación (laravel/ui)	<code>login</code> , <code>logout</code> , <code>register</code> , <code>password.request/e-mail/reset/update/confirm</code> , <code>verification.notice/verify/resend</code>	guest / auth / signed / throttle	<code>verification.verify</code> redirige a <code>/admin</code>
Sugerencias	<code>sugerencias.send</code>	ninguno (valida usuario verificado en el controlador)	Envía <code>SugerenciaMail</code>
Panel	<code>admin.index</code>	auth, verified	Tablero y calendario
<code>admin.usuarios.*</code> , <code>admin.administracion.*</code>	8 + 8	auth, verified, can:*	Solo admin

Grupo	Rutas (nombre base)	Middleware	Observaciones
admin.alumnos.*, admin.areas.*, admin.gestores.*, admin.horarios.* (+ cargar_datos_areas)	8 + 8 + 8 + 9	auth, verified, can:*	admin y administrativo
admin.cursos.* (+ calendar-events)	8 + 1	auth, verified, can:*, (calendar-events solo auth)	Grupo con prefijo admin
admin.asignaciones.*	7 (claves compuestas {gestor}/{curso}, sin show)	auth, verified, can:*	Valida solapamiento de horario
admin.proyectos.*, admin.presupuestos.*	8 + 7 (presupuestos sin show)	auth, verified, can:*	
admin.proyecto_gestores. *, admin.alumno_proyecto.*	7 + 7	auth, verified, can:*, (edit/update de proyecto_gestores sin can : → P-02)	Nombres confirm-delete
inscripciones.*	create, store, mis-cursos, destroy, mis_proyectos	auth, verified, can:*	alumno y usuario; mis_proyectos solo alumno
reportes.*	index, cursos-asignados, mis_proyectos	auth, verified, can:*	gestor
post.*	create, store, notifications (/post)	auth, verified, can:*/ auth	Avisos y bandeja
Notificaciones	markAsRead (GET), markNotification (POST /mark-as-read)	auth	Marcar leídas; JSON en AJAX
API	GET /api/user	auth:sanctum	Sin uso funcional

16. Anexo D: Referencia Rápida de Comandos

```
# Ciclo de vida
docker compose up -d --build           # construir y levantar
docker compose ps / logs -f            # estado y registros
docker compose down                     # apagar conservando datos

# Aplicación
docker compose exec app php artisan migrate --seed
docker compose exec app php artisan db:seed --class=DemoSeeder
docker compose exec app php artisan optimize:clear
docker compose exec app php artisan test
docker compose exec app php artisan route:list --except-vendor
docker compose exec app php artisan tinker

# Base de datos
docker compose exec db psql -U ncie -d ncie
docker compose exec -T db pg_dump -U ncie -Fc ncie > ncie_$(date +%F).dump
docker compose exec -T db pg_restore -U ncie -d ncie --clean --if-exists < ncie_<fecha>.dump

# Recursos estáticos y permisos
docker compose --profile assets run --rm node
docker compose exec app chmod -R ug+rw storage bootstrap/cache public/uploads/fotos
```